DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

Academic Year 2026-2027 -ODD SEMESTER

Skill-Based Evaluation Report

(III Internal Assessment)

for

23AI2001

ARTIFICIAL INTELLIGENCE FOR CYBER SECURITY

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE)

Submitted by

Nitish (URK24CS5030)
Alston Renz (URK24CS5052)
Ferdinand Garson (URK24CS5068)
Fevin (URK24CS5067)
Jeffrey Joshua (URK24CS5065)
Venkat Gonuguntla (URK24CS5048)

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the skill-based evaluation report for the course **23AI2001 Artificial Intelligence for Cyber Security** is a bonafide work done during the even semester of the academic year 2026-2027 by

Nitish (Reg.No: URK24CS5030)

Alston Renz (Reg.No: URK24CS5052)

Ferdinand Garson (Reg.No: URK24CS5068)

Fevin (Reg.No: URK24CS5067)

Jeffrey Joshua (Reg.No: URK24CS5065)

Venkat Gonuguntla (Reg.No: URK24CS5048)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence) of Karunya Institute of Technology and Sciences.

_____

Submitted for the assessment held on

Signature of Course Instructor
Mr. HARIVIGNESH K S
Assistant Professor

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

S. No.	Evaluation Parameter	Max. Marks	Marks Secured
1	Technical Proficiency and Project Implementation	10	
2	Problem-Solving Ability	5	
3	Use of AI Models and Techniques	5	
4	Project Output and Result Analysis	5	
5	Project Report, PPT and Viva Voce	10	
6	Online Certification	5	
	Total	**40**	

Signature of Course In-charge
Mr. HARIVIGNESH K S
Assistant Professor

Table of Contents

Abstract 4

1. Introduction 4

1.1 Background 4
1.2 Motivation 5
1.3 Scope of the Project 5
1.4 Organisation of the Report 5

2. Literature Review 5

2.1 Intrusion Detection Systems 5
2.2 Machine Learning for Intrusion Detection 5
2.3 Datasets and Evaluation Practice 6
2.4 Explainability in Security Applications 6
2.5 Summary of Related Work 6

3. Problem Statement 7

4. Objectives 7

5. System Overview and Workflow 8

5.1 Data Input and Validation 9
5.2 Preprocessing and Prediction 9

6. Machine Learning Models 10

6.1 Random Forest 10
6.2 Logistic Regression 10

7. Explainable AI with SHAP 11

7.1 Global Importance 11
7.2 Local Explanations 11
7.3 Interpreting Explanations Responsibly 11

8. Dashboard Features 12

9. Evaluation Metrics and Methodology 16

9.1 The Confusion Matrix 16
9.2 Precision, Recall and F1-score 16
9.3 Why Recall Matters in Intrusion Detection 16
9.4 Testing Methods 17

10. Results and Discussion 17

10.1 Random Split 17
10.2 Wednesday Holdout 17
10.3 Comparison of the Two Models 18
10.4 Possible Reasons for the Gap 18
10.5 Implications 19

11. Limitations and Future Improvements 19

11.1 Broader Validation 20
11.2 Reliability and Reproducibility 20
11.3 Responsible Use 20

12. Conclusion 20

References 20

Abstract

Networks generate very large volumes of traffic data, and only a fraction of that traffic is suspicious. Reviewing such data manually is slow, and security teams need predictions that they can also understand. This report presents **SentinelAI**, an academic prototype for AI-powered network intrusion detection built on the CICIDS2017 dataset (2,830,743 flow records described by 78 numeric features). The system accepts network traffic data in CSV format, validates and preprocesses it, and classifies each record as either **BENIGN** or **MALICIOUS** using two machine learning models: Random Forest and Logistic Regression. A second, separately trained Random Forest then names the likely attack type for records flagged as malicious, or reports the type as uncertain when its confidence is low.

To make the predictions easier to trust and inspect, SentinelAI uses SHAP (SHapley Additive exPlanations) to show which features influence a prediction, both across a set of records and for one individual record. A Streamlit dashboard that runs entirely on the local machine brings together data upload, prediction with severity-ranked alerts, model evaluation, explanations, and a local prediction history stored in SQLite.

Evaluation shows that the models perform strongly under a random train-test split, where Random Forest reaches an F1-score of **0.995** with a false-positive rate of 0.13%. When tested on held-out traffic from a different day, however, recall drops sharply: 20.0% for Random Forest (22.0% without class weighting) and 32.9% for Logistic Regression, which in turn raises false alarms on 6.0% of benign traffic. The report reviews the related literature, describes the system and its evaluation, discusses this gap, and sets out the limitations and improvements needed before real-world use.

1. Introduction

1.1 Background

Modern networks continuously produce records of the traffic flowing through them. Most of this traffic is benign, but some of it may be suspicious or malicious. Intrusion detection is the task of separating the two so that security teams can respond to genuine threats. The idea of detecting intrusions automatically from audit records goes back several decades [1], and machine learning is now widely studied for this purpose because it can learn patterns from labelled traffic records and apply them to new data at a speed that manual review cannot match [2], [3].

1.2 Motivation

Manually reviewing large traffic datasets takes time, and analysts cannot realistically inspect every record. A classifier can narrow the effort by flagging records that look malicious. A prediction alone, however, is not always enough: security teams also need to understand why a model reached its decision. This is the motivation for combining classification with explainable AI in SentinelAI.

1.3 Scope of the Project

SentinelAI is an academic prototype built to analyse network traffic datasets supplied as CSV files. It uses the public CICIDS2017 MachineLearningCSV files and is designed to run on an ordinary laptop with 8 GB of RAM, by reading the data in chunks and training on a bounded sample. It is intended to demonstrate the full path from uploaded data to a classified, explained result. It analyses stored or uploaded records and does not capture live traffic. It is not a production security product, and the report is explicit about the limitations that currently prevent real-world deployment.

1.4 Organisation of the Report

Chapter 2 reviews related work on intrusion detection, machine learning, datasets, and explainability. Chapters 3 and 4 state the problem and the objectives. Chapter 5 describes the system workflow, Chapter 6 the two machine learning models, Chapter 7 the use of SHAP, and Chapter 8 the dashboard. Chapter 9 explains the evaluation metrics and methodology, Chapter 10 presents the results and discusses them, Chapter 11 lists limitations and future improvements, and Chapter 12 concludes the report.

2. Literature Review

2.1 Intrusion Detection Systems

An intrusion detection system monitors activity and raises an alert when it appears to be malicious. Early work proposed modelling normal behaviour and treating significant deviations as possible intrusions [1]. Network-based systems apply this idea to network traffic. They are commonly divided into signature-based approaches, which look for patterns of known attacks, and anomaly-based approaches, which look for departures from normal traffic. Signature-based methods are precise for known attacks but cannot recognise new ones, while anomaly-based methods can flag unfamiliar behaviour but may raise more false alarms [2], [3].

2.2 Machine Learning for Intrusion Detection

Several surveys describe how machine learning and data mining have been applied to intrusion detection, covering decision-tree ensembles, linear models, support vector machines, and deep

neural networks [2], [3], [4]. In the supervised setting, each traffic record carries a label, and a model is trained to predict that label for new records. Ensemble methods such as Random Forest are popular because they can capture non-linear relationships among many numeric features [5], while Logistic Regression remains a common, easily understood baseline [6]. These are the two models used in SentinelAI.

2.3 Datasets and Evaluation Practice

The quality of an evaluation depends heavily on the data. Ring et al. survey the publicly available network-based intrusion detection datasets and compare their properties [7]. Some of these datasets are collected over several days, with different traffic and attacks on different days [8], which makes it possible to test a model on a day that was not used for training. SentinelAI uses one such dataset, CICIDS2017 [8], which covers five working days of benign traffic mixed with denial-of-service, distributed denial-of-service, port-scan, brute-force, web, botnet and infiltration attacks. About 80.3% of its records are benign, and Monday contains only benign traffic.

Sommer and Paxson argued that machine learning for intrusion detection is harder than in many other fields, because errors are costly, network traffic is highly varied, and realistic evaluation is difficult [9]. More recently, Arp et al. catalogued common pitfalls in security-related machine learning, including sampling bias, data snooping, and evaluation that does not reflect real deployment, and showed that such pitfalls can make results look better than they really are [10]. These pitfalls are relevant to the gap between random-split and holdout results reported in Chapter 10.

2.4 Explainability in Security Applications

Because security decisions carry consequences, there is growing interest in models whose behaviour can be examined. Doshi-Velez and Kim argue for a rigorous science of interpretability [11], and Guidotti et al. survey methods for explaining black-box models [12]. Well-known methods include LIME, which fits a simple local model around one prediction [13], and SHAP, which assigns each feature a contribution based on Shapley values [14]. SHAP has also been extended with efficient algorithms for tree-based models, which is relevant to Random Forest [15]. SentinelAI adopts SHAP for these reasons.

2.5 Summary of Related Work

Table 1 summarises how each area relates to SentinelAI.

Table 1: Related work and relevance to SentinelAI

Area	Representative work	Relevance to SentinelAI
Intrusion detection concepts	Denning [1]; Buczak and Guven [2]	Frames intrusion detection as separating normal from malicious activity.
ML and DL for IDS	Liu and Lang [3]; Ahmad et al. [4]	Shows the range of models used; supports a two-model comparison.
Model foundations	Breiman [5]; Hosmer et al. [6]	Basis for the Random Forest and Logistic Regression models.
Datasets	Ring et al. [7]; Sharafaldin et al. [8]	Explains why day-based holdout testing is possible and informative.
Evaluation pitfalls	Sommer and Paxson [9]; Arp et al. [10]	Helps interpret the gap between random-split and holdout results.
Explainable AI	Ribeiro et al. [13]; Lundberg and Lee [14]; Lundberg et al. [15]	Basis for using SHAP to explain predictions.

3. Problem Statement

Network environments produce more traffic data than analysts can review by hand. Some of that traffic is benign while some may be malicious, and the two cannot be told apart quickly without automated support. The problem addressed by this project is therefore how to build a system that:

- accepts network traffic data and checks that it has the expected structure;
- predicts whether each traffic record is benign or malicious;
- explains which features influenced a prediction so that the result can be inspected; and
- reports honestly how well the models perform, including on traffic they have not seen before.

4. Objectives

SentinelAI is designed around three core objectives, supported by three further goals, summarised in Table 2.

Table 2: Project objectives

Objective	Description
Analyze	Accept network traffic data in CSV format.
Classify	Predict whether each record is BENIGN or MALICIOUS.
Explain	Use SHAP to show which features influence predictions.
Evaluate	Review model evaluation results and compare the models.
Record	Keep a local history of predictions.
Identify	Name the likely attack type for malicious records, or mark it as uncertain.

5. System Overview and Workflow

SentinelAI follows a simple pipeline that takes a user from an uploaded file to a result with explanations. Figure 1 shows the five stages a user sees, Table 3 describes what happens at each one, and Figure 2 shows the full architecture, including the offline stage that trains and saves the models.

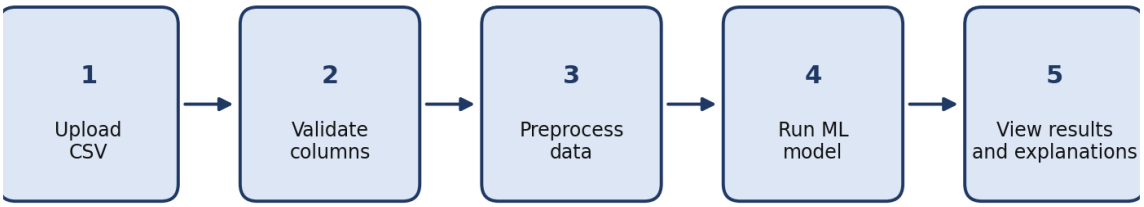


Figure 1: SentinelAI system workflow

Table 3: Workflow stages

Step	Stage	Description
1	Upload CSV	The user uploads a file of network traffic records in CSV format.
2	Validate columns	The application checks that the expected feature columns are present.
3	Preprocess data	The data is prepared into the form the models expect.
4	Run ML model	The selected model classifies each record as BENIGN or MALICIOUS.
5	View results and explanations	Predictions, evaluation results, and SHAP explanations are shown in the dashboard.

The architecture has two parts. The offline part runs once from the command line: the eight CICIDS2017 files (2,830,743 records) are read in chunks of 20,000 rows, column names are

standardised, and textual infinity values are converted to missing values. A bounded, deterministic sample of at most 3,000 records per class per file (40,160 records in total) keeps training within the memory of an 8 GB laptop. The models are trained and evaluated on this sample, and the fitted pipelines and evaluation reports are saved to disk. The online part is the Streamlit dashboard, which loads the saved models, checks each uploaded file, runs the binary and attack-type models, and presents alerts, explanations and stored history.

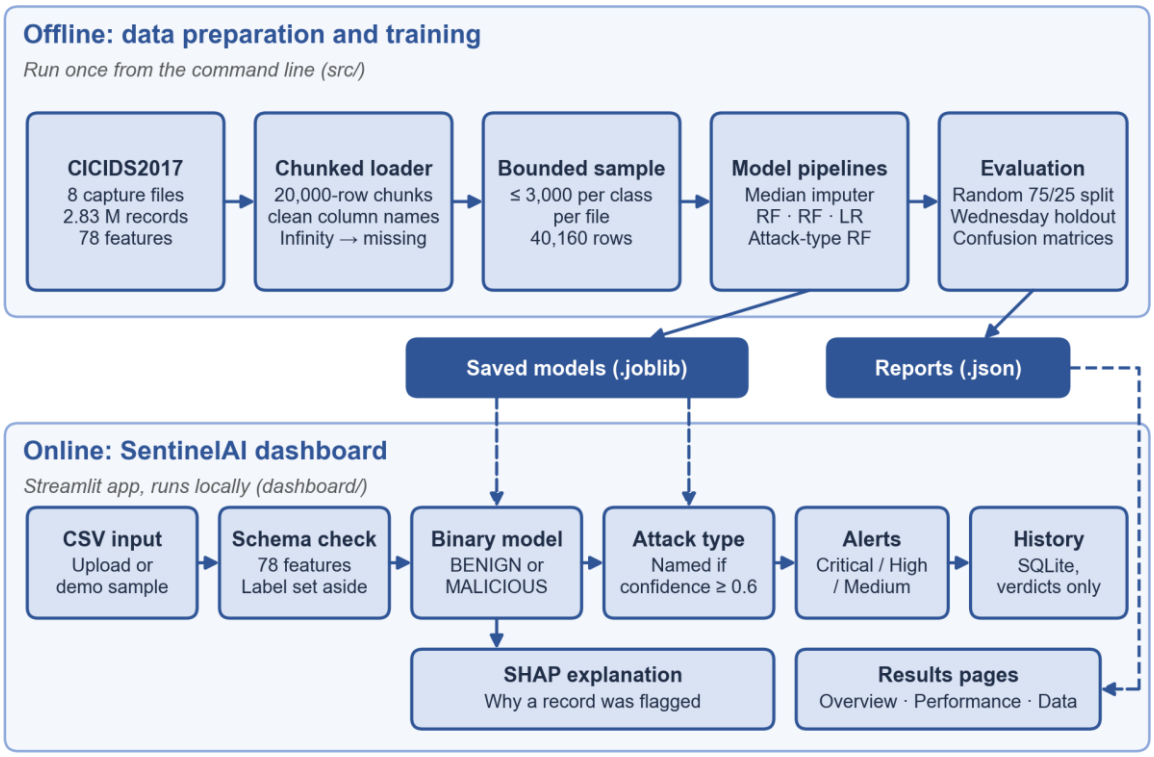


Figure 2: SentinelAI system architecture

5.1 Data Input and Validation

Input is supplied as a CSV file of up to 500 MB, or the built-in demo sample of 1,811 labelled Wednesday records can be used. Column names are standardised (for example, Flow Bytes/s becomes flow_bytes_s), and the duplicated Fwd Header Length column in the raw files is kept under a distinct name. The application then checks that exactly the 78 expected feature columns are present; missing or unexpected columns are reported by name and the file is rejected. An optional Label column is kept aside for comparison and is never passed to the model. This step protects the models from files that do not match the structure they were trained on, and it gives the user early feedback when an upload is unsuitable.

5.2 Preprocessing and Prediction

Validated data passes through the same preprocessing that was fitted during training, because each saved model is a complete scikit-learn pipeline. Missing values, including former infinity

values, are replaced by the median learned from the training records only, a missingness indicator is added for each affected feature, and for Logistic Regression the features are also standardised. Each traffic record then receives a label of BENIGN or MALICIOUS together with an attack score. Records labelled MALICIOUS are passed to the attack-type model, which names one of 11 supported attack types when its confidence is at least 0.60 and otherwise reports the type as uncertain. Simple, transparent rules combine the two scores into Critical, High or Medium severity. The results, together with the evaluation metrics and explanations, are then presented in the dashboard.

6. Machine Learning Models

Two model types are used for traffic classification. Both use the 78 dataset features to predict whether a record is BENIGN or MALICIOUS, and using two different model types allows their behaviour to be compared. All models are trained on the same sample with a fixed random seed (42), so the results can be reproduced.

6.1 Random Forest

Random Forest is an ensemble method that combines the predictions of many decision trees. Because each tree can split the feature space differently, the ensemble can learn complex patterns in network traffic features, and combining many trees reduces the risk of relying on any single tree [5]. SentinelAI uses 120 trees, considers the square root of the number of features at each split, and requires at least two records per leaf. The main variant uses balanced_subsample class weighting, which gives the smaller attack class more weight in each tree; an unweighted variant is kept as a controlled comparison. A separate Random Forest with 160 trees is trained only on attack records to name the attack type. It covers the 11 attack labels with at least 100 records; Heartbleed (11 records), Infiltration (36) and SQL injection (21) are too rare to learn and are reported as uncertain rather than guessed.

6.2 Logistic Regression

Logistic Regression estimates class probabilities using a weighted combination of the input features. It is a simpler, more linear model than Random Forest, and it provides a useful baseline against which the more complex model can be compared [6]. In SentinelAI it is trained with balanced class weights and the liblinear solver on standardised features.

Table 4: Comparison of the two models

Aspect	Random Forest	Logistic Regression
Approach	Combines predictions from multiple decision trees.	Estimates class probabilities from a weighted combination of features.
Strength	Can learn complex patterns in traffic features.	Simple and straightforward, useful as a baseline.
Output	BENIGN or MALICIOUS	BENIGN or MALICIOUS
Configuration	120 trees, square-root feature sampling, at least two records per leaf, balanced_subsample weights.	Balanced class weights, liblinear solver, standardised features.

7. Explainable AI with SHAP

A classifier that only returns a label gives analysts little to work with. SentinelAI therefore uses SHAP, which estimates how much each feature contributes to a model prediction [14]. For Random Forest the efficient tree algorithm is used [15], and for Logistic Regression the linear explainer.

7.1 Global Importance

Global importance summarises the influence of each feature across the records being explained. It shows which features matter most to the model overall for the data that was uploaded. In the dashboard it is averaged over a bounded sample of 50 analysed records so that it can be computed in a few seconds.

7.2 Local Explanations

A local explanation shows how the features influence one individual prediction. This helps an analyst see why a particular record was labelled BENIGN or MALICIOUS. The dashboard shows the ten most influential features for a chosen record: red bars push the prediction towards MALICIOUS and teal bars towards BENIGN. For a DoS GoldenEye record in the demo sample, for example, the destination port and the timing between forward packets pushed most strongly towards MALICIOUS (Figure 5).

7.3 Interpreting Explanations Responsibly

SHAP explanations must be read with care. Two points are stressed throughout the project:

- An explanation describes model behaviour; it does not prove that the prediction is correct.
- Feature influence depends on the uploaded data and on the trained model, so the same feature may matter differently in a different dataset.

8. Dashboard Features

The application is a local Streamlit dashboard with six pages, grouped in the sidebar under Detect, Review and Data and described in Table 5. The active model is chosen once in the sidebar and applies to every page. Figures 3 to 6 show the main pages.

Table 5: Dashboard components

Component	Purpose
Overview	Summarise the dataset and the selected model's results on both tests, with the key finding in one sentence.
Analyze traffic	Run the models on an uploaded CSV or the demo sample; review alerts by severity, a summary and all records, with CSV export.
Explain a prediction	Explore SHAP feature contributions for any analysed record, filtered by attacks or normal records.
Model performance	Compare the three models on both tests, including the trade-off between attacks caught and false alarms.
Prediction history	Store and review saved predictions locally with SQLite; only verdicts are stored, never traffic data.
Dataset explorer	Describe each capture file and attack type, including the types that are too rare to learn.

Figure 3 shows the Overview page. It summarises the dataset in four figures and places the selected model's results on familiar traffic beside its results on an unseen day, so the most important finding of the project is visible on the first screen.



Figure 3: Overview page of the SentinelAI dashboard

Figure 4 shows the Analyze traffic page after running the demo sample with Random Forest. Of 1,811 records, 973 were flagged as attacks and 698 of those as Critical, and the predictions agreed with the true labels for 81.2% of records. Where the attack-type model is not confident, as for the Heartbleed record in row 6, the type is shown as uncertain instead of being guessed.

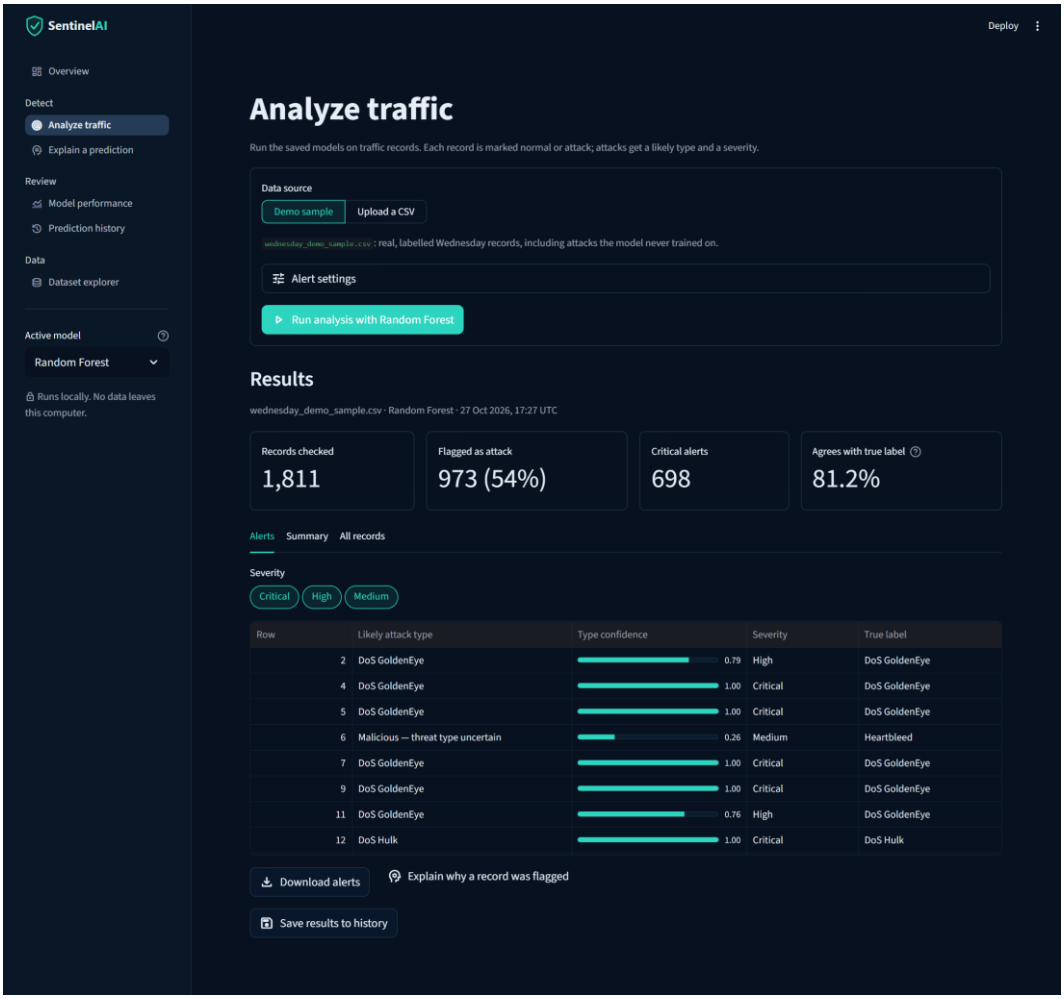


Figure 4: Analyze traffic page after running the demo sample

Figure 5 shows the Explain a prediction page, where any analysed record can be selected and its local SHAP explanation is drawn as a bar chart.

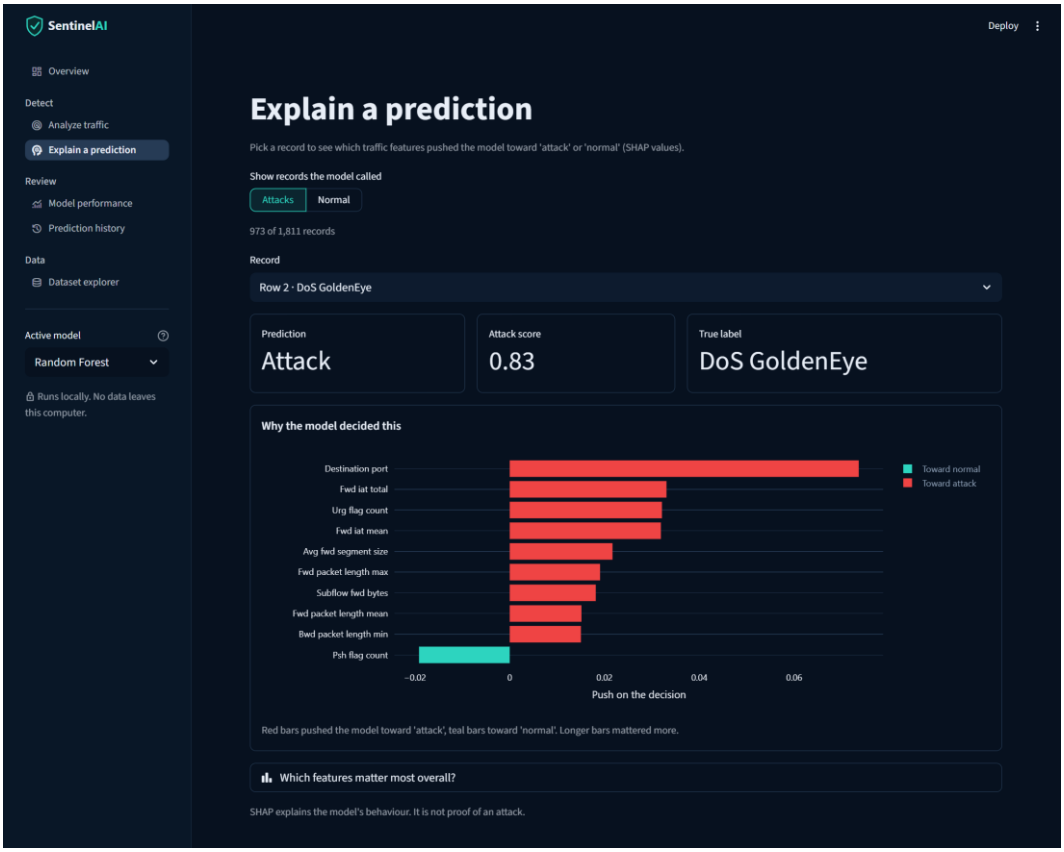


Figure 5: Explain a prediction page with a local SHAP explanation

Figure 6 shows the Model performance page on the Wednesday holdout test. The four counts on the right are the confusion matrix of the selected model written in plain language.

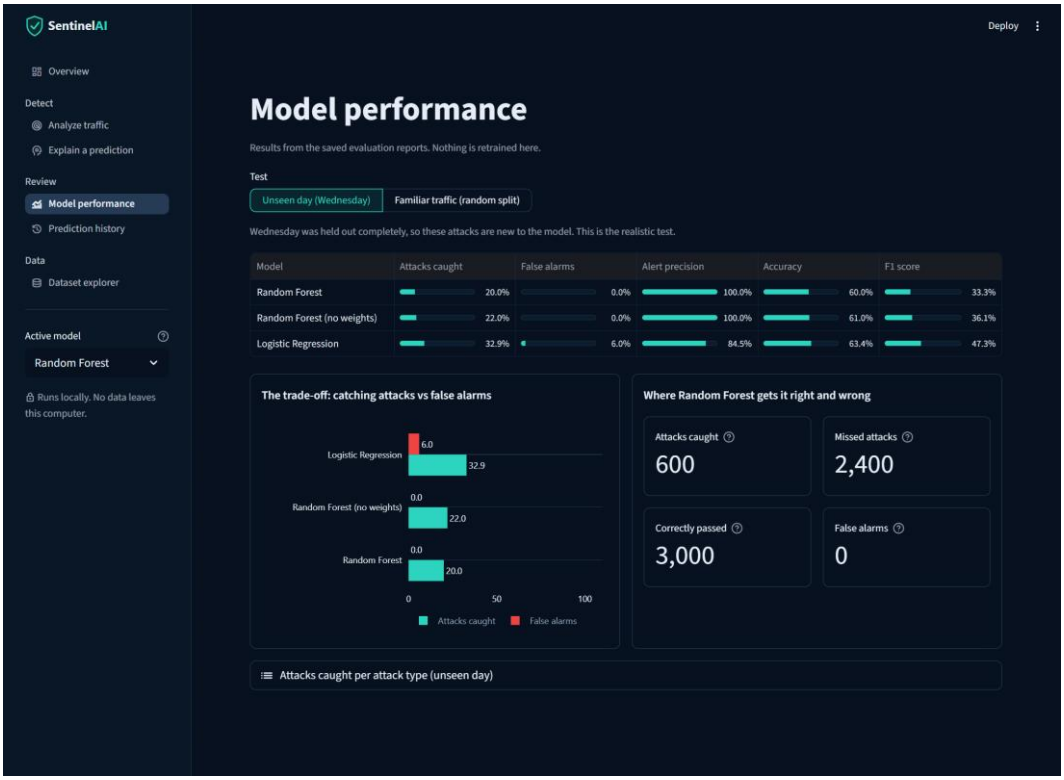


Figure 6: Model performance page on the Wednesday holdout test

9. Evaluation Metrics and Methodology

Before the results are presented, this chapter explains the metrics used to judge a classifier and the two testing methods applied in the project.

9.1 The Confusion Matrix

For a binary classifier, every prediction falls into one of four outcomes. Treating MALICIOUS as the positive class, these are:

- **True positive (TP):** a malicious record correctly flagged as malicious.
- **False positive (FP):** a benign record wrongly flagged as malicious, which produces a false alarm.
- **True negative (TN):** a benign record correctly recognised as benign.
- **False negative (FN):** a malicious record wrongly labelled benign, which means a missed detection.

9.2 Precision, Recall and F1-score

Four metrics are derived from these counts and are summarised in Table 6.

Table 6: Evaluation metrics

Metric	Formula	What it tells us
Precision	TP / (TP + FP)	Of the records flagged as malicious, how many really were.
Recall	TP / (TP + FN)	Of all the malicious records, how many were found.
F1-score	2 × Precision × Recall / (Precision + Recall)	A single value that balances precision and recall.
False-positive rate	FP / (FP + TN)	Of all the benign records, how many were wrongly flagged as malicious.

9.3 Why Recall Matters in Intrusion Detection

In intrusion detection, a false negative means an attack passes unnoticed, whereas a false positive costs analyst time. Recall therefore shows directly how much malicious traffic the system misses. A model can have a high overall score and still miss many attacks if the data is dominated by benign traffic, which is why recall on malicious traffic is reported separately in the holdout test.

9.4 Testing Methods

SentinelAI is evaluated with two methods. In a **random split**, records are shuffled and divided into training and test sets, so both sets come from the same overall distribution. In a **holdout** test, all records from one day (Wednesday) are kept aside and the models are trained on the rest. The holdout method is closer to real use, because a deployed system always meets traffic that was not available during training. Evaluating only on randomly split data is one of the pitfalls that can make results look better than they are [9], [10].

10. Results and Discussion

How a model is tested strongly affects the result that is reported. The difference between the two testing methods is one of the most important findings of the project.

10.1 Random Split

Under a random split (75% training and 25% testing, stratified by class), training and test records come from similar distributions. In this setting the weighted Random Forest reaches an accuracy of 99.61%, a precision of 99.80%, a recall of 99.23% and an F1-score of **0.995**, with a false-positive rate of 0.13%: only 8 of 6,000 benign test records were flagged and 31 of 4,040 attacks were missed. Logistic Regression is weaker on this test (F1-score 0.894, false-positive rate 11.2%). A result like this shows that the models can separate the classes well when the test data resembles the training data.

10.2 Wednesday Holdout

In the holdout test, all Wednesday records (3,000 benign and 3,000 malicious in the sample) are kept aside and not used for training, so the models are tested on traffic they have not seen. Performance drops considerably. Random Forest catches 600 of the 3,000 attacks (recall **20.0%**), the unweighted variant 22.0%, and Logistic Regression 986 (recall **32.9%**), as shown in Figure 7. Most Wednesday attacks in the sample are DoS GoldenEye (2,799 of 3,000), and Random Forest detected only 19.2% of them.

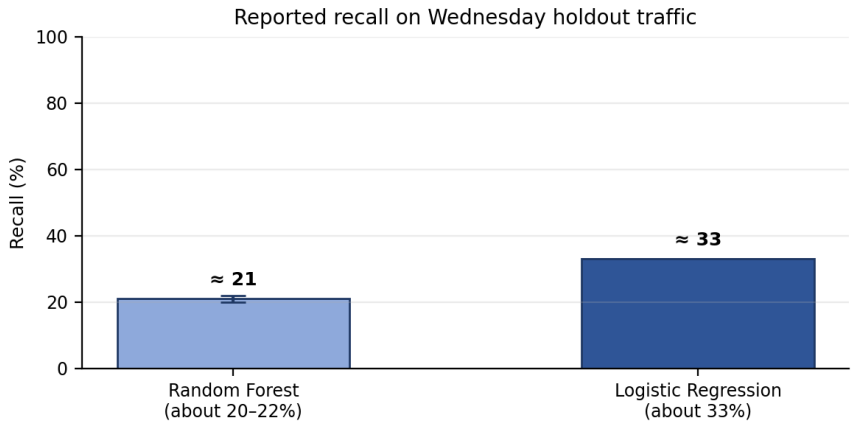


Figure 7: Reported recall on Wednesday holdout traffic (approximate)

Table 7: Summary of evaluation results

Test method	Reported result	Interpretation
Random split	Random Forest: accuracy 99.6%, F1-score 0.995, false-positive rate 0.13%	Strong when training and test records have similar distributions.
Wednesday holdout	Recall: Random Forest 20.0% (unweighted 22.0%); Logistic Regression 32.9%	Performance drops on held-out traffic.
Wednesday holdout	Precision: Random Forest 100% (no false alarms); Logistic Regression 84.5% (181 false alarms)	Higher recall comes at the cost of false alarms.
Attack-type model (25% holdout)	Macro F1-score 0.93; weighted F1-score 0.97	Strong for common types; weaker for Web Attack XSS (recall 50%) and Brute Force (74%).

10.3 Comparison of the Two Models

On the holdout traffic, Logistic Regression reached a higher recall (32.9%) than Random Forest (20.0%), but at a cost: it raised 181 false alarms out of 3,000 benign records (false-positive rate 6.0%, precision 84.5%), whereas Random Forest raised none (precision 100%). The two models therefore sit at different points of the trade-off between catching attacks and avoiding false alarms. Both recall values are low, so neither model can be considered reliable for unseen traffic, and class weighting changed Random Forest's recall by only two percentage points.

10.4 Possible Reasons for the Gap

The project reports the gap between the two tests but does not claim to have established its cause. Several explanations are plausible and could be examined in future work:

- The held-out day may contain traffic or attack behaviour that differs from what the models saw in training, a general problem for learned detectors [9].
- A random split can place very similar records, for example records from the same period or session, in both the training and test sets, which makes the test easier than real use [10].
- A model may rely on features that are specific to the training data and do not carry over to a different day, which SHAP explanations could help to investigate.

10.5 Implications

The contrast between the two tests shows that a very high score on a random split should not be taken as evidence that the system will detect threats in new traffic. The low recall on held-out traffic means that many malicious records were missed. This is why the project reports both results together and treats broader validation as a priority. In an operational setting, the choice of model and decision threshold would have to be made on separate validation data, according to how many missed attacks and false alarms a security team can accept.

11. Limitations and Future Improvements

The prototype has several known limitations. Each is paired with the improvement that would address it in Table 8.

Table 8: Limitations and improvements

Limitation	Improvement
Large uploads may be limited to the first 100,000 rows.	Make this behaviour explicit to the user.
Performance on held-out traffic days is weak.	Validate more broadly across separate days of traffic.
Multiclass threat-type scores are not yet tested across separate capture days or files.	Test multiclass scores across separate capture days or files.
Only scikit-learn is pinned (1.7.2, matching the saved models); other dependencies are not.	Pin every dependency to an exact version.
The 19 automated tests cover schema checks, row numbering, history and model loading, but not every edge case.	Add tests for oversized files, malformed CSVs, missing model files, and prediction consistency.

11.1 Broader Validation

The most important improvement is to test the models on more than one held-out day, so that the weak holdout recall can be confirmed, explained, and improved. Testing multiclass threat-type scores across separate capture days or files belongs to the same effort.

11.2 Reliability and Reproducibility

Only scikit-learn is currently pinned, at version 1.7.2 to match the saved models; pinning every dependency would make results reproducible on another machine. The current automated test suite (19 tests) passes. Tests for oversized files, malformed CSVs, missing model files, and prediction consistency would make the application more robust, and making the upload row limit explicit would remove a source of confusion for users.

11.3 Responsible Use

SentinelAI is a learning prototype and should not be the only safeguard for a real network. Missed detections are a serious risk, and an explanation of a prediction does not make the prediction correct. Any wider use would first require the validation described above.

12. Conclusion

SentinelAI is an academic prototype for network traffic classification on the CICIDS2017 dataset. It combines a memory-conscious data pipeline, two binary classifiers and a separate attack-type model, SHAP explanations, and local prediction history in a single Streamlit dashboard. Evaluation shows near-perfect results under a random split (F1-score 0.995) but much weaker recall on traffic from an unseen day (20.0% for Random Forest and 32.9% for Logistic Regression). The main lesson of the project is that how a model is tested matters as much as the model itself. More robust validation across several held-out days, clear upload limits, and fully reproducible testing are needed before the system could be considered for real-world use.

References

[1] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, 1987.

[2] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.

[3] H. Liu and B. Lang, "Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey," Applied Sciences, vol. 9, no. 20, Art. 4396, 2019.

[4] Z. Ahmad, A. Shahid Khan, C. Wai Shiang, J. Abdullah and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," Transactions on Emerging Telecommunications Technologies, vol. 32, no. 1, e4150, 2021.

[5] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[6] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ: Wiley, 2013.

[7] M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, "A survey of network-based intrusion detection data sets," Computers & Security, vol. 86, pp. 147–167, 2019.

[8] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116.

[9] R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305–316.

[10] D. Arp et al., "Dos and Don'ts of Machine Learning in Computer Security," in Proc. 31st USENIX Security Symposium, 2022.

[11] F. Doshi-Velez and B. Kim, "Towards A Rigorous Science of Interpretable Machine Learning," arXiv:1702.08608, 2017.

[12] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti and D. Pedreschi, "A Survey of Methods for Explaining Black Box Models," ACM Computing Surveys, vol. 51, no. 5, Art. 93, 2018.

[13] M. T. Ribeiro, S. Singh and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765–4774.

[15] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67, 2020.

Certificate